

AUDITORÍA TÉCNICA • 2026-09-30

flywheel: análisis y propuestas de refactor del plugin

OBJETO

repo	arazvan-ec/xmarks
versión	0.83.0
commit	5249626
superficie	18 skills • 6 agentes • 66 scripts

MÉTODO

- 01 sweep propio ejecutado (48 pasos)
- 02 shellcheck sobre 12.604 líneas de bash
- 03 cuatro auditorías paralelas por dominio
- 04 contraste con agent-development, writing-skills, skill-creator y claudes-automation-recommender

Veredicto

El repo está sano. No es un caso de arreglar lo roto.

El riesgo dominante es el **coste marginal de cada cambio futuro** —masa y acoplamiento—, no un defecto presente. Todo lo que el repo se propone verificar, lo verifica; lo que no tiene es una forma barata de seguir creciendo.

Las tres cifras de la derecha se midieron ejecutando el repo, no leyéndolo.

48/48

Sweep completo en verde. 13 gates de CI más 35 suites de test, ejecutados en este análisis.

0 errores

shellcheck sobre 12.604 líneas. Los 3 warnings son falsos positivos, revisados uno a uno.

3 gates

Paridad vigilada. Hooks, agentes y sweep ↔ CI no pueden divergir en silencio.

La evidencia de salud

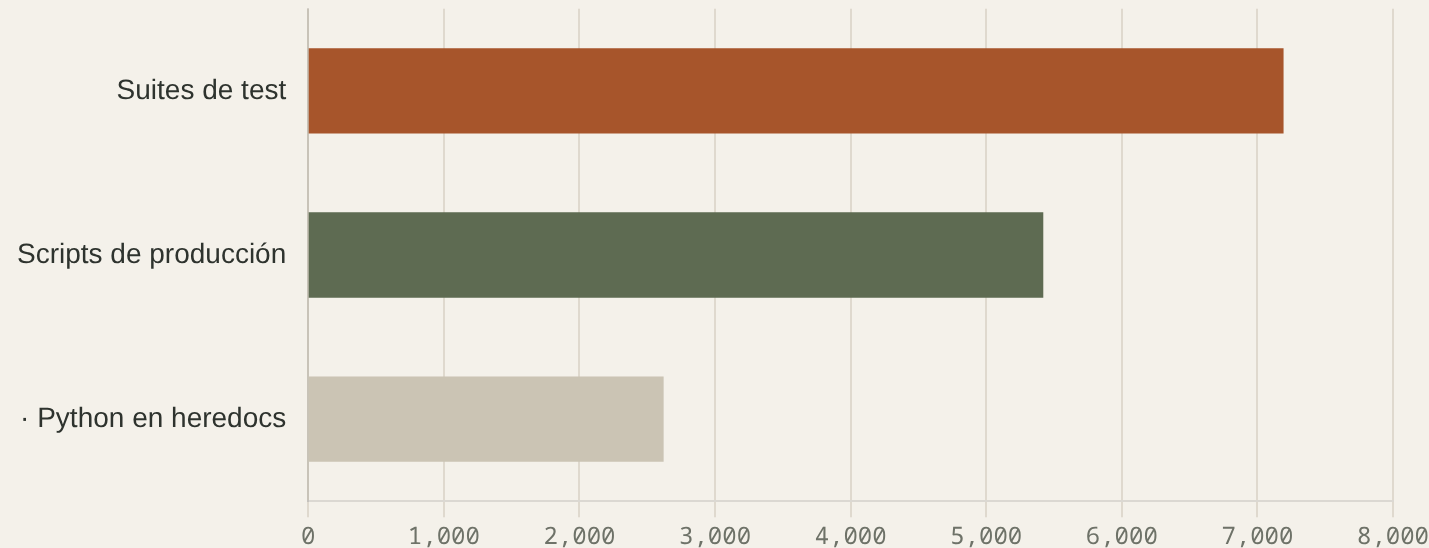
No es una impresión: son seis señales medidas ejecutando el repositorio en su commit actual.

SEÑAL	MEDICIÓN	CÓMO SE VERIFICA
Sweep completo 13 gates de CI + 35 suites de test	48/48 PASS	bash scripts/sweep.sh
Lint de shell 66 scripts, 12.604 líneas de bash	0 errores 3 warnings	shellcheck -S warning scripts/*.sh
Los 3 warnings, revisados uno a uno	falsos positivos	SC1007 sobre VAR= cmd , un prefijo de entorno legítimo
Paridad de hooks y agentes lo distribuido frente a lo autoaplicado	vigilada	check-hook-parity.sh check-agent-parity.sh
Paridad sweep local ↔ CI no hay lista paralela que mantener	vigilada	check-ci-gate-parity.sh lee los run: del propio YAML
Presupuesto de descriptions de skills	3538/3600	check-description-budget.sh

Ejecutado en sandbox el 2026-09-30 sobre 5249626 · `claude plugin validate --strict` quedó SKIPPED por ausencia del CLI local, único paso no reproducido.

Dónde está el riesgo real: la masa

REPARTO DE LAS 12.604 LÍNEAS DE BASH



Las 2.616 líneas de Python viven dentro de los scripts de producción: son un subconjunto, no una cuarta pila. Fuente: `wc -l` sobre `scripts/` en 5249626.

LA LECTURA

Cada cifra es trabajo hecho. Juntas son el **precio de cada cambio futuro**: nada de esto está roto, y todo encarece la siguiente línea.

5-26 % de las líneas de test es **aserción real**; el resto es arnés repetido

34× **pass()** definido de nuevo, casi byte a byte, en cada suite

80 notas de upgrade en **90 días** — un release al día, sin CHANGELOG

Retorno inmediato 1: sweep paralelo

ESFUERZO S · MEDIDO, NO ESTIMADO

HOY · SECUENCIAL

1m18s

for t in scripts/test-*.sh
35 suites, una detrás de otra

CON XARGS -P4

0m20s

mismo resultado · 0 fallos
verificado en este análisis

3,9×

de espera devuelta al
desarrollador en cada
ciclo

Por qué ya es seguro

- ✓ Cada test crea su propio `mktemp -d` con `trap` de limpieza.
- ✓ Ninguno hace `cd` al árbol compartido del repo.
- ✓ Los que tocan ficheros de presupuesto escriben en **copias bajo WORK**, nunca en el original.

Dónde está el tiempo

- 🕒 `test-eval-graders.sh` — 13,1 s
- 🕒 `test-fixture-scratch.sh` — 8,7 s
- 🕒 `test-install-vendored.sh` — 8,1 s

Retorno inmediato 2: shellcheck y supply chain

ESFUERZO S · DOS GATES

El repo es 100 % bash y no tiene linter de shell

Ningún workflow ejecuta shellcheck. Y como **hoy pasa limpio**, el gate entra en verde el primer día: no hay deuda que pagar antes de instalarlo, solo deuda que impedir.

El caso raro en que añadir un gate **no cuesta nada** y a partir de ahí la calidad no puede degradarse sin que CI lo diga.

Lo que además falta

concurrency:	ausente en los 4 workflows: dos pushes casi simultáneos pueden correr release.yml en paralelo sobre tags
cache: npm	el CLI se reinstala entero en cada corrida
dependabot	no existe, con 2 acciones externas en juego

El pinning protege un workflow y deja tres fuera

check-supply-chain-pin.sh blinda con rigor el caso crítico. Los otros tres siguen en tag móvil — y uno de ellos es justo el pipeline que **valida el propio plugin**.

```
flywheel-update.yml:64 checkout@11bd719...
flywheel-update.yml:161 create-pull-request@271a8d0...

release.yml:36 checkout@v4
validate-plugins.yml:29 checkout@v4
validate-plugins.yml:31 setup-node@v4
validate-plugins.yml:50 checkout@v4
```

Un tag **v4** re-apuntado es el vector clásico contra GitHub Actions. El estándar que el repo se exige en un sitio no se aplica en los otros tres.

Defectos latentes, todos de esfuerzo S

NINGUNO ROMPE CI HOY

01 • SIGPIPE

El bug ya reproducido, corregido a medias

El commit **38a7a1e** lo documentó con datos —**5/300 fallos falsos**, y era lo que enrojecía check-task-closure de forma intermitente— pero el fix solo tocó el fichero de test. El patrón sigue vivo en producción.

```
check-fixture-leaks.sh:77 • renumber.sh:55
grep -q alimentado por pipe bajo pipefail
```

02 • Convención

La regla de set es tácita

El diseño es correcto —los hooks deben fallar abiertos— pero **nada lo declara ni lo verifica**: el próximo hook puede nacer con **set -e** y bloquear un turno por un fichero ausente.

```
8 scripts sin ninguna línea set
17 con -uo pipefail • 6 con -euo pipefail
read-meter.sh con solo -u
```

03 • Instalador

Temporales huérfanos y glob sin red

Con **set -euo pipefail** pero **sin ningún trap**, una instalación que falla a medias deja ficheros en /tmp. Y sin **nullglob**, un directorio vacío pasaría el glob literal como nombre de fichero.

```
install-vendored.sh:339 y :358 (mktemp)
install-vendored.sh:264 y :395 (glob)
```

04 • Límite 128 KiB

Cuatro hooks truncan en silencio

read-meter.sh usa fichero temporal precisamente porque documenta que el payload puede exceder el límite de variable de entorno. Los otros cuatro pasan el JSON por **env** y heredan ese límite.

```
bash-allow.sh • write-allow.sh
delegation-guard.sh • read-prime.sh
```

Producto: la description contamina el disparador

writing-skills documenta con un caso reproducido que si la description resume el flujo, el agente **sigue la description y no lee el body**. En ese caso hizo una revisión en vez de dos.

El riesgo concreto aquí: **review** describe el enrutamiento por tipo de diff en su propia description — así que un agente puede decidir el fan-out **sin leer la lógica real**.

LOS CUATRO PEORES

loop	enumera las 6 fases del ciclo
work	el inner loop completo, paso a paso
run	los 5 pasos del contrato
review	el enrutamiento y la síntesis

LA CORRECCIÓN, SOBRE EL CASO DE loop

HOY — RESUME EL WORKFLOW

«...spec → plan → work → verify → review → compound, gating between phases, with a tight inner micro-cycle. Use when...»

PROPUESTO — SOLO DISPARADOR

«Use when starting a new feature or task and you want the full gated cycle instead of running one phase manually.»

El gate existente **check-description-budget.sh** mide cuánto pesa (3538/3600 chars), no de qué tipo es: una description puede cumplir el presupuesto y violar la regla. Falta una heurística que exija «Use when» al inicio y marque los encadenamientos de proceso.

Agentes y hooks: lo que falta y lo que se pisa

Agentes

6 ficheros
agents/*.md

Falta un campo que la guía marca como requerido

Ninguno declara `color:`, ni tiene la sección **When to invoke** en el cuerpo. El campo propio `effort:` es una extensión legítima, pero no sustituye a `color`.

El menor privilegio está en la prosa, no en el frontmatter

Los tres `reviewer-*` declaran `Bash` mientras su system prompt dice «**Do not modify files**». Bash permite `sed -i`: la restricción no se aplica sola.

Hooks

3 en Stop
1 en cada tool call

Tres bloqueadores independientes en Stop

`gate.sh`, `toolbar.sh` y `compound-due.sh` hacen `exit 2` por razones no relacionadas: suite roja, formato de la barra, compound pendiente.

`read-meter.sh` corre en cada llamada

Con matcher `.*`: arranca CPython y relee el log completo cada vez. **Reproduce a nivel de hooks el coste cuadrático del que advierte.**

El modelo recibe los motivos **fragmentados en reintentos sucesivos**: arregla el formato, reintenta, ahora falla el compound, reintenta, ahora falla el gate. Un orquestador que agregue los tres motivos en un solo mensaje convierte tres ciclos en uno, a coste cero.

Los dos refactors de mayor apalancamiento

ESFUERZO L · INCREMENTALES

A · TESTS

scripts/lib/fwtest.sh

Un arnés común, porque hoy la mayoría de cada suite es plomería y la deriva ya empezó: **dos semánticas distintas de «test rojo»** conviven en el repo.

- 34× `pass()` redefinido, casi byte a byte
- 33× `fail()` redefinido
- 7× `run_hook()` reimplementado
- 3× graders de evals **idénticos byte a byte**

RIESGO HONESTO

Un arnés roto rompe **41 suites a la vez**. Mitigación en tres capas: self-test del arnés que corre **primero** en el sweep, migración empezando por los 7 `run_hook` —el grupo más homogéneo— y **nunca big-bang**.

B · PRODUCCIÓN

fw_common.sh + fw_allowlist.py

Sacar la lógica del heredoc: **2.616 líneas de Python** viven como texto dentro de bash, así que no son lintables ni testeables unitariamente.

- 3 implementaciones **distintas** del parser de allow-list, con semánticas divergentes
- 8 copias del patrón `SKIP_<GATE>`
- 7 copias de la resolución de `CLAUDE_PROJECT_DIR`
- 996 líneas en los **4 heredocs mayores**, las primeras candidatas a extraer

EL PRECEDENTE ES PROPIO

`fw_cutoffs.py` y `fw_tasks.py` ya están extraídos e invocados como CLI desde `check-route-honored.sh:64`. **El patrón funciona en este repo; solo falta aplicarlo al resto.**

Documentación y estado acumulado

El README mezcla tres audiencias en el mismo párrafo

Referencia de uso, rationale de diseño y bitácora de versiones, intercalados línea a línea. **11 párrafos de más de 150 palabras** y 6 de los 19 encabezados llevan la versión en el título.

Buena parte documenta el pasado, no el estado actual

README:152 – «It used to git clone this repo's main and bash the result...»

README:132 – «Before it, 14 of 16 shipped runs carried no compound line.»

Son notas de migración disfrazadas de referencia, y ya existen mejor contadas en [upgrades/](#) y [docs/research/](#).

La rotación del ledger está diseñada y nunca se construyó

[docs/research/git-native-memory-design.md:115-121](#) especifica el archivado por trimestres. El directorio no existe. Y **LEARNINGS.md se inyecta en cada sesión**: es el producto que vende disciplina de tokens creciendo sin rotación.

LAS CIFRAS

5.431

palabras de README, con **31 menciones de versión**

1.049

líneas de **LEARNINGS.md** sin política de archivado

1, 2 MB

de estado en [.claude/flywheel/](#), 133 ficheros versionados

0

CHANGELOG, con **80 notas de upgrade** ya escritas

Hoja de ruta: doce specs, tres olas

CADA UNA CON SU GATE

P72–P74
días · riesgo
casi nulo

Lo que se cobra solo

- P72** Sweep paralelo, 3,9× medido — `test-sweep.sh`
- P73** shellcheck en CI, pin SHA ×4, concurrency, cache, dependabot — `check-supply-chain-pin.sh`
- P74** Convención de `set`, `fix SIGPIPE` ×2, `trap` y `nullglob`, límite 128 KiB — `check-shell-conventions.sh`

P75–P78
comportamiento

Lo que cambia cómo se comporta el producto

- P75** Rotación de `LEARNINGS.md` · **P76** `descriptions` = disparador
- P77** `color`, `When to invoke` y menor privilegio en los 6 agentes
- P78** Consolidar los tres hooks `Stop` en un orquestador

P79–P83
incremental

El gran refactor, nunca de golpe

- P79** `fwtest.sh` con `self-test` · **P80** `fw_common.sh` y los 4 heredocs
- P81** `README` → `docs/` y `CHANGELOG` generado · **P82** descomponer el instalador
- P83** Subagente `bash-reviewer` enrutado por `diff`

QUÉ NO TOCAR

La disciplina de gates. 13 gates con test propio es infraestructura cara de construir y barata de mantener.

El sistema de paridad. Cierra la clase de bug más difícil de ver con dos rutas de distribución.

Los presupuestos versionados. Retunear es un cambio de datos revisable, no de código.

La honestidad sobre las evals. Admitir que las katas son `regression-only` vale más que un número bonito.

El modelo de confianza de `gate.sh`. Hash fuera del repo y re-consentimiento al editar.

Ninguna propuesta pide abandonar el método del repo: cada una llega con el gate que la mantiene viva.